

GOT4Rec: Graph of Thoughts for Sequential Recommendation

Zewen Long^{1,2}, Liang Wang^{1,2}, Shu Wu^{1,2}, Qiang Liu^{1,2}, Liang Wang^{1,2}

¹NLPR, MAIS, Institute of Automation, Chinese Academy of Sciences

²School of Artificial Intelligence, University of Chinese Academy of Sciences

{zewen.long, liang.wang}@cripac.ia.ac.cn, {shu.wu, qiang.liu, wangliang}@nlpr.ia.ac.cn

Abstract

With their vast open-world knowledge and reasoning abilities, large language models (LLMs) have become a promising tool for sequential recommendation. Researchers have explored various methods to harness these capabilities, but most existing approaches rely on simple input-output prompting, failing to effectively bridge the gap between LLMs’ general knowledge and the specific needs of recommendation tasks. While reasoning strategies like chain-of-thought (CoT) have been introduced to enhance performance, they often produce inaccurate recommendations due to underutilized user preference information and insufficient reasoning depth. To address these challenges, we propose GOT4Rec, a novel sequential recommendation method leveraging the graph of thoughts (GoT) reasoning strategy. Our method focuses on three key types of information in user histories: short-term interests, long-term interests and collaborative information from other users. It enables LLMs to reason independently and generate recommendations, subsequently aggregating results to derive final items. This method allows LLMs, with enhanced reasoning capabilities, to better utilize the user sequence information, producing more accurate recommendations and comprehensive explanations. Extensive experiments on real-world datasets demonstrate the effectiveness of GOT4Rec, outperforming existing state-of-the-art baselines with an average improvement of 37.11%. Our code is available at <https://anonymous.4open.science/r/GOT4Rec>.

1 Introduction

Sequential recommendation has long been a significant research field, with numerous methods proposed to explore chronological dependencies within user sequences [Kang and McAuley, 2018; Zhou *et al.*, 2020]. Despite considerable advancements, they remain constrained by the limited knowledge available from datasets. To overcome this, it is crucial to integrate real-world knowledge into sequential recommendation models, enabling them to more effectively comprehend

and reason about preference patterns within user behavior sequences [Hou *et al.*, 2022; Li *et al.*, 2023a].

Large language models (LLMs) have recently attracted significant attention for their expansive open-world knowledge and advanced reasoning abilities. Consequently, numerous LLM-based sequential recommendation methods [Xi *et al.*, 2024; Sanner *et al.*, 2023; Hou *et al.*, 2024b] have emerged, aiming to capture user interests from interaction sequences while leveraging LLMs’ comprehensive real-world knowledge for recommendations. Many of these approaches rely solely on the input-output prompting paradigm, which underutilizes the reasoning potential of LLMs. As a result, it often produces task-irrelevant or inaccurate outcomes. To address this problem, several studies have explored advanced reasoning strategies to boost LLMs’ performance in sequential recommendation. One example is SLIM [Wang *et al.*, 2024], a knowledge distillation module which employs chain-of-thought (CoT) [Wei *et al.*, 2022] prompting to enable step-by-step reasoning in sequential recommendation, transferring knowledge from a teacher model to a student model. However, it only utilizes the basic reasoning abilities of LLMs. This is insufficient for effectively processing the rich preference information embedded in user sequences, such as long-term and short-term interests or spatio-temporal patterns. Consequently, it often produces inaccurate recommendations. As illustrated in Figure 1, SLIM’s CoT-based reasoning focuses narrowly on predicting snack bars, while the true preference is a fruit nut mix, highlighting its limitations in handling diverse preference structures.

Therefore, the challenge lies in integrating diverse user preferences into a cohesive reasoning framework, turning sequential recommendation into a multi-faceted task requiring advanced reasoning capabilities. Studies have shown that simple approaches like CoT are inadequate for tasks that demand decomposition into sub-tasks [Besta *et al.*, 2024; Yang *et al.*, 2024]. In contrast, the graph of thoughts (GoT) method offers a more effective solution by decomposing the problem into more manageable components. Unlike CoT, which employs a single chain of thought, GoT models the reasoning process through networked reasoning. This approach allows GoT to break down the complex user preference reasoning tasks into smaller, more tractable sub-tasks, solve them individually, and then integrate the results to form a comprehensive solution. As illustrated in Figure 1, GoT



Figure 1: Comparison of the LLM output and predictions for the next item generated by SLIM and our proposed GOT4Rec.

reasons over multiple product categories that the user might be interested in: Probiotic Snack Bars, Healthy Snack Mixes and Low-Sugar, Low-Carb Cookies, enabling the LLM to independently generate recommendations within each category and combine them to correctly predict the ground truth item: fruit nut mix. While SLIM’s recommendation remains focused on snack bars, showcasing its narrower approach.

To address the challenges mentioned above, this paper proposes GOT4Rec, a novel method that introduces the graph of thoughts (GoT) framework into the field of sequential recommendation. GOT4Rec optimally harnesses the reasoning capabilities of LLMs while effectively integrating multiple sources of information from user sequences. Specifically, we employ the GoT reasoning strategy to extract three critical sources of information from user sequences: short-term interests, long-term interests, and collaborative interests from other users with similar preferences. Unlike previous methods that treat the sequence as a whole, our approach considers multiple aspects of information and better utilizes the reasoning abilities of LLMs, leading to more accurate and interpretable recommendations. Our key contributions can be summarized as follows:

- To the best of our knowledge, this work is among the first to explore the application of the graph of thoughts framework in the context of sequential recommendation.
- We introduce GOT4Rec, which optimally leverages the reasoning capabilities of LLMs to comprehensively capture and utilize the information within user sequences.
- Extensive experiments conducted on three datasets demonstrate that our method outperforms neural and LLM-based sequential models as well as other LLM reasoning strategies.

2 Related Work

2.1 Sequential Recommenders

Traditional neural sequential recommendation systems aim to capture sequential dependencies in user behavior sequences to model dynamic user preferences, as seen in methods like GRU4Rec [Hidasi *et al.*, 2016]. Techniques like self-attention and graph neural networks have further advanced the field [Kang and McAuley, 2018; Zhou *et al.*, 2020; Wu *et al.*, 2019; Zhang *et al.*, 2022; Zhang *et al.*, 2023]. These methods however, predominantly rely on sequence modeling capabilities and often inadequately incorporate textual information. Recently, research on transferable item representations [Hou *et al.*, 2022; Li *et al.*, 2023a] has gained attention. These approaches remain constrained by limited datasets and fail to fully leverage real-world knowledge. With the advent of LLMs, new potential are offered by leveraging LLMs’ language understanding for recommendations. They can enhance features [Xi *et al.*, 2024; Lin *et al.*, 2024; Liu *et al.*, 2024] or act as rankers [Dai *et al.*, 2023; Harte *et al.*, 2023; Li *et al.*, 2023b; Sanner *et al.*, 2023; Hou *et al.*, 2024b]. For example, ReLLa [Lin *et al.*, 2024] employs semantic user behavior retrieval to improve data quality and introduces retrieval-enhanced instruction tuning to enhance few-shot recommendation. LLMRank [Hou *et al.*, 2024b] formalizes the recommendation problem as a conditional ranking task and utilizes LLMs as zero-shot rankers. Although these approaches are promising, existing research has not yet fully capitalized on the reasoning capabilities of LLMs.

2.2 LLM Reasoning Strategies

Numerous approaches have been proposed to exploit the reasoning capabilities of LLMs. Chain-of-thought (CoT) [Wei *et al.*, 2022] introduces intermediate reasoning steps to enhance LLM performance, while Chain of Thought with Self-Consistency (CoT-SC) [Wang *et al.*, 2023] refines this by generating multiple CoTs and selecting the best output. Tree of thoughts (ToT) [Yao *et al.*, 2024] and graph of thoughts (GoT) [Besta *et al.*, 2024] further extend these methods by modeling reasoning as a tree or graph, respectively, to better generate and aggregate different thoughts, thereby improving complex tasks. In the recommendation domain, SLIM [Wang *et al.*, 2024] utilizes a CoT-based knowledge distillation module to transfer the step-by-step reasoning capabilities from a teacher model to a student model. However, its reliance on basic CoT limits its ability to fully capture the abundant sequential dependencies and intricate patterns within user interactions.

3 The Proposed GOT4Rec Method

In this section, we introduce GOT4Rec, a sequential recommendation method that fully leverages the reasoning capabilities of LLMs to capture the diverse information within user sequences. In the context of sequential recommendation, given a user u ’s historical interaction sequence $S_u = \{i_1, i_2, \dots, i_{n-1}\}$, the task is to predict the next item i_n that the user is most likely to interact with.

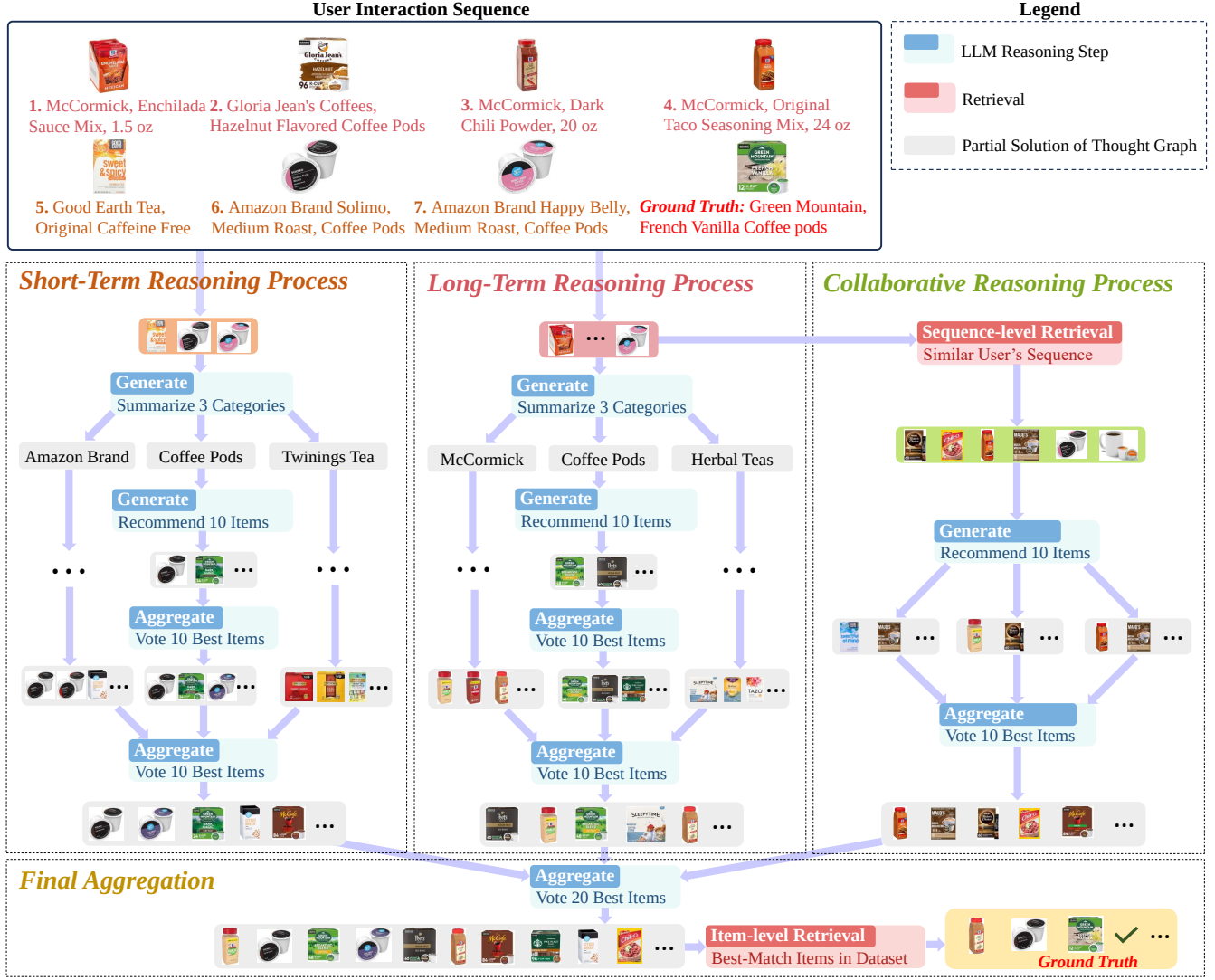


Figure 2: An example of graph decomposition in the proposed GOT4Rec method. The user interaction sequence is divided to facilitate different types of reasoning: the last few items are used for short-term preference reasoning, while the entire sequence informs long-term preference reasoning. Additionally, sequences from similar users are retrieved for collaborative reasoning. In the thought graph, these thoughts are generated and subsequently aggregated to capture and integrate the various aspects of information in the user interaction sequence.

3.1 Overview of Thought Graph

When interacting with LLMs, we input messages (prompts) and LLMs respond with generated outputs (thoughts). Building on the GoT framework [Besta *et al.*, 2024], we model our GOT4Rec method as a tuple (G, T) , where G represents the reasoning process and T denotes the thought transformations. Specifically, we model the reasoning process as a directed graph $G = (V, E)$, where V is the set of vertices and $E \subseteq V \times V$ is the set of edges. Each vertex represents a thought, encapsulating a solution to the current recommendation step along with other relevant global information. A directed edge (t_1, t_2) signifies a dependency between thoughts, indicating that thought t_2 is generated by LLMs based on t_1 .

In our method, we employ the generation transformation T_G and aggregation transformation T_A from GoT framework.

The generation transformation generates one or more new thoughts based on an existing thought v . In this operation, new vertices and edges are generated: $V_+ = \{v_{1+}, \dots, v_{k+}\}$ and $E_+ = \{(v, v_{1+}), \dots, (v, v_{k+})\}$, where $v_{1+}, \dots, v_{k+}$ are new thoughts generated by v . Reasoning steps such as CoT can be incorporated into this process. While the aggregation transformation combines multiple thoughts into a consolidated thought, reinforcing their strengths while mitigating their weaknesses. A new vertex v_+ is created in this operation: $V_+ = \{v_+\}$ and $E_+ = \{(v_1, v_+), \dots, (v_k, v_+)\}$, where $v_1, \dots, v_k$ are the aggregated thoughts.

Figure 2 provides an example of graph decomposition in our GOT4Rec method. In the recommendation pipeline, the current user's interaction sequence S_u is utilized as input. We then generate three key aspects of user preference informa-

Summarizing Prompt: To summarize categories/brands that the user prefers.

I've purchased the following products in the past in order: <Item Sequence>. Please help me do the following things:
 Step 1: Could you help me identify the key factors that influence my choice of products by analyzing my purchase history (summarize my preferences briefly)? Let's work this out in a step by step way to be sure we have the right answer.
 Step 2: Could you help me select three product **Categories** or **Brands** that appeal to me the most based on my personal preferences?
 In step 2, please recommend the categories or brands directly and split these output with a line break (Format: no. a product category or brand).

Recommendation Prompt: To recommend items for user.

I've purchased the following products in the past in order: <Item Sequence>. Based on my purchase history, your task is to recommend 10 products from Amazon that best fit the <Category/Brand>. Please only generate the name of the products and split these output with a line break (Format: no. a product, Example: 1. a product).

Collaboration Prompt: Recommend items based on other user's sequence.

I've purchased the following products in the past in order: <Item Sequence>. There are several users who share the same interests with me, please help me do the following things :
 Step 1: Could you help me identify the key factors that influence our choice of products by analyzing the purchase history of me and the other users(summarize our preferences briefly)? Let's work this out in a step by step way to be sure we have the right answer.
 Step 2: Based on our purchase history, your task is to recommend 10 products from Amazon that best fit our interests, you can choose products from other users' purchase histories.
 In step 2, please only generate the name of the products and split these output with a line break (Format: no. a product). The purchase history of other users are as follows: <\$co> .

Voting Prompt: Let LLM vote for the best items.

I've purchased the following products in the past in order: <Item Sequence>. Given a set of products, your task is to vote 10 best products in the set that best fit my purchase history. Here is the set of products:<Recommended Items>.

Figure 3: Prompt templates, including summarizing, recommendation, collaboration and voting prompts.

tion: short-term preference, long-term preference and collaborative preference, with detailed definitions provided in the subsequent subsection. With these preference information, the LLMs are enabled to reason and generate a list of top- N items that best align with the user's current interests. Finally, these items are aggregated, and the LLMs vote to select the most probable items for recommendation.

3.2 Short-Term & Long-Term Reasoning Process

Numerous studies have emphasized the importance of understanding both a user's dynamic short-term and stable long-term preferences in recommendation systems [Yu *et al.*, 2019; Zheng *et al.*, 2022]. In our GOT4Rec method, we enable LLMs to extract these two types of preferences separately and then synergize the most relevant aspects of each. To reason about short-term preferences, we select the last few interactions from the user sequence S_u as S_{short} . We then apply and enhance the zero-shot CoT strategy from [Wang *et al.*, 2024] within the generation transformation T_G to effectively capture the user's short-term preferences and identify three categories that the user is likely to favor. As illustrated in Figure 3 as summarizing prompt, the generation transformation T_G involves two key steps:

- Step 1. Summarize user's short-term preferences based on the given S_{short} .
- Step 2. Based on the preferences in step 1, summarize three categories of products the user is inclined to prefer.

For each identified category, we prompt the LLMs to generate N items the user is most likely interested in, repeating this process three times. The prompt template is provided in Figure 3 as recommendation prompt. After generating the item sets, we utilize the aggregation transformation T_A , where the three item sets undergo a voting process by the LLMs. This results in a new set of N items that are most likely to interest the user within each category. After aggregation, we have three final item sets, which are subjected to a final round of voting by the LLMs to determine the top- N items I_{short} that best represent the user's short-term preferences. The voting prompt is also provided in Figure 3.

To capture long-term preferences, we extend the input to include the entire user interaction sequence. Similar to the short-term reasoning process, we employ the zero-shot CoT strategy within the generation transformation T_G , as shown in Figure 3. This allows LLMs to effectively differentiate and identify short-term and long-term preferences. After generating the relevant items, the final set of top- N items I_{long} , representing the user's long-term preferences, is determined through the aggregation transformation T_A .

3.3 Collaborative Reasoning Process

Collaborative filtering is widely used in various recommendation scenarios, modeling users' preference based on their past interactions. In our method, we leverage the all-mpnet-base-v2 [Reimers and Gurevych, 2020], a sentence-transformers model that maps sentences to a vector space, to generate an embedding vector for the current user's interaction sequence, allowing us to retrieve a set of sequences S_{co} from other users most similar to the current user's sequence. Details of this retrieval process are provided in the following section.

Once we have obtained the similar user sequences S_{co} , we employ the zero-shot CoT strategy to generate three sets of top- N items that the current user may be interested in. The two-step generation prompts, illustrated in Figure 3 as collaboration prompt, are as follows:

- Step 1. Summarize the shared preferences between the current user and other users based on the given S_{co} .
- Step 2. Based on the summarized preferences from Step 1, recommend N items selected from S_{co} that the current user is likely to prefer.

The three sets of items generated through this process are then aggregated and selected by the LLMs using the aggregation transformation T_A . This results in the final set of top- N items I_{co} that best reflect the collaborative preferences of users with similar interests to the current user.

At this point, we have obtained three distinct sets of items: I_{short} , which reflects the user's short-term preferences; I_{long} , which captures the user's long-term preferences; and I_{co} , which represents the collaborative preferences derived from other users. Finally, using the aggregation transformation T_A , we enable LLMs to synthesize these multiple sources of information to generate the final set of top- N recommended items I_{fin} . The prompt template for this final aggregation is the same as the voting prompt.

3.4 Multi-Level Retrieval Module

To identify other users' sequences that share same interests with the current user and to assess how closely the recommended items match the ground truth in sampled datasets, we employ a two-level retrieval approach.

Sequence-level Retrieval. As previously mentioned, when analyzing collaborative preferences, it is essential to retrieve sequences of other users who share similar interests with the current user. We choose to deploy the Mpnet [Reimers and Gurevych, 2020] model f_{mpnet} due to its superior efficiency and discriminative power in encoding item titles compared to other encoding models such as BERT. Given a query sequence $S_u = \{i_1, \dots, i_m\}$, the retrieved sequences set $\mathcal{S}_{\mathcal{V}_{\text{seq}}}^k$ can be obtained as follows:

$$\mathcal{S}_{\mathcal{V}_{\text{seq}}}^k(i_t) \triangleq \arg \max_{v \in \mathcal{V}_{\text{seq}}} \frac{1}{m} \sum_{t=1}^m \text{sim}(f_{\text{mpnet}}(i_t), v) \quad (1)$$

where $\mathcal{S}_{\mathcal{V}_{\text{seq}}}^k$ is the set of top- k sequences that share similar interests with the current user. $\text{sim}(\cdot, \cdot)$ denotes the computation of Euclidean distance, $\mathcal{V}_{\text{seq}}$ represents the vector base containing the embeddings of all other sequences.

Item-level Retrieval. The title of an item generated by LLMs may differ from the title of the ground truth in datasets, even though they refer to the same item. This discrepancy arises due to the limited and varied versions of items in the datasets (e.g., *Witcher 3: Wild Hunt* versus *Witcher 3: Wild Hunt Complete Edition*). To address this issue, we continue to utilize the f_{mpnet} model to encode item titles into vectors. We then retrieve the most similar items from the vector base by computing the inner product of the query vector and all other vectors in the base. Specifically, for an item i_{query} generated by LLMs, the retrieval process is as follows:

$$\mathcal{I}_{\mathcal{V}_{\text{item}}}^k(i_{\text{query}}) \triangleq \arg \max_{v \in \mathcal{V}_{\text{item}}} \text{sim}(f_{\text{mpnet}}(i_{\text{query}}), v) \quad (2)$$

where $\mathcal{I}_{\mathcal{V}_{\text{item}}}^k$ represents the set of the retrieved top- k items based on $\text{sim}(\cdot, \cdot)$, which denotes the computation of the inner product and $\mathcal{V}_{\text{item}}$ is the vector base containing all item embeddings. In practice, we observed that in most cases, when the description of the query item is vague, the retrieved items are misaligned (e.g., when querying for *The Witcher 3*, the top results include [*Diablo III*, *The Witch and the Hundred Knight - PlayStation 3*, *The Witcher 3: Wild Hunt - Xbox One*]). To mitigate this, we retrieve the top- K items within the ranked indices for each recommended item.

3.5 Inference Latency and Volume Analyzation

In Table 1, we compare the inference lantency and volume between reasoning frameworks. Lantency is the number of hops to reach the final thought and volume refer to the number of preceding thoughts that have impacted the final thought [Besta *et al.*, 2024]. The time to generate a single thought is assumed to be $O(1)$. In CoT, reasoning follows a single sequential chain, resulting in both latency and volume scaling with N . CoT-SC employs k independent chains, noting that these chains support parallel execution, both are reduced to N/k . For a k -ray ToT tree, both latency and volume are

$\log_k N$. As for GoT, it is a k -ray tree combined with its inverted counterpart, resulting in a latency of $\log_k N$. For volume, since all intermediate thoughts contribute to the final thought by aggregating information from every step, it scales with N .

Framework	Latency	Volume
Chain-of-Thought (CoT)	N	N
Multiple CoTs (CoT-SC)	N/k	N/k
Tree of Thoughts (ToT)	$\log_k N$	$\log_k N$
Graph of Thoughts (GoT)	$\log_k N$	N

Table 1: Comparison of inference latency and volume for different reasoning frameworks.

Overall, by empowering LLMs to independently reason and generate thoughts and then aggregate the most relevant results, our method fully leverages the reasoning capabilities of LLMs in the sequential recommendation scenario. This approach enables LLMs to effectively capture and integrate different aspects of the extensive information within user sequences.

4 Experiments

4.1 Experimental Settings

Datasets. We conduct experiments on three item categories from Amazon Reviews'23 dataset [Hou *et al.*, 2024a]: Video Games (**Games**), Grocery and Gourmet Food (**Food**) and Home and Kitchen (**Home**). Reviews are treated as user-item interactions, sequenced chronologically by timestamps. We focus on users with 6 to 20 interactions and filter out items with fewer than five interactions. Following [Wang *et al.*, 2024], we randomly sample 3,000 users from each dataset three times. We employ the leave-one-out strategy for dataset division: the most recent interaction for testing, the second most recent interaction for validation and the remaining are used for training. Both training and validation segments are used as input during testing.

Baselines. To demonstrate the effectiveness of our proposed method, we select two groups of recommendation baselines. The first group comprises both neural and LLM-based sequential recommenders:

- **SASRec** [Kang and McAuley, 2018]: A self-attention based sequential model to capture user's preferences.
- **PALR** [Yang *et al.*, 2023]: A novel framework that integrates user behavior with LLMs to generate personalized recommendations.
- **TALLRec** [Bao *et al.*, 2023]: An efficient fine-tuning framework that aligns LLMs with recommendation systems through a two-stage tuning process.

The second group contains models that utilize LLM reasoning strategies:

- **SLIM** [Wang *et al.*, 2024]: A knowledge distillation module transferring the step-by-step reasoning capabilities in recommendation from a larger teacher model to a smaller student model.

Dataset	Metric	SASRec	PALR	TALLRec	SLIM	CoT	CoT-SC	ToT	GOT4Rec	Improv.
Games	HR@5	<u>0.0776</u>	0.0630	0.0660	0.0602	0.0644	0.0653	0.0489	0.0894	15.21%
	HR@10	0.0953	0.0893	0.0967	0.0977	0.0988	<u>0.1013</u>	0.0712	0.1167	15.20%
	HR@20	0.1227	0.1157	0.1233	0.1281	<u>0.1347</u>	0.1253	0.1087	0.1361	1.04%
	NDCG@5	<u>0.0528</u>	0.0385	0.0401	0.0380	<u>0.0408</u>	0.0421	0.0304	0.0621	17.61%
	NDCG@10	<u>0.0586</u>	0.0423	0.0497	0.0502	0.0519	0.0537	0.0377	0.0710	21.16%
	NDCG@20	<u>0.0655</u>	0.0475	0.0562	0.0578	0.0609	0.0622	0.0471	0.0760	16.03%
Food	HR@5	0.0335	0.0337	0.0390	0.0350	0.0396	<u>0.0443</u>	0.0273	0.0742	67.49%
	HR@10	0.0407	0.0487	0.0513	0.0517	0.0581	<u>0.0597</u>	0.0390	0.0972	62.81%
	HR@20	0.0549	0.0657	0.0747	0.0613	0.0748	<u>0.0753</u>	0.0533	0.1090	44.75%
	NDCG@5	<u>0.0286</u>	0.0216	0.0243	0.0216	0.0253	<u>0.0276</u>	0.0182	0.0492	72.03%
	NDCG@10	0.0309	0.0235	0.0284	0.0270	0.0311	<u>0.0326</u>	0.0219	0.0567	73.93%
	NDCG@20	0.0337	0.0267	0.0318	0.0295	0.0352	<u>0.0366</u>	0.0255	0.0597	63.11%
Home	HR@5	0.0132	0.0130	0.0123	0.0123	0.0118	<u>0.0133</u>	0.0047	0.0192	44.36%
	HR@10	0.0179	0.0167	0.0183	0.0180	0.0177	<u>0.0223</u>	0.0100	0.0299	34.08%
	HR@20	0.0262	0.0213	0.0233	0.0213	0.0230	<u>0.0270</u>	0.0147	0.0337	24.81%
	NDCG@5	<u>0.0097</u>	0.0080	0.0081	0.0073	0.0073	0.0080	0.0031	0.0122	25.77%
	NDCG@10	0.0105	0.0094	0.0096	0.0091	0.0093	<u>0.0109</u>	0.0049	0.0157	44.04%
	NDCG@20	<u>0.0134</u>	0.0101	0.0113	0.0099	0.0106	0.0121	0.0060	0.0167	24.63%

Table 2: Recommendation performance. The best performance is highlighted in **bold** and the runner-up is highlighted by underlines. Improvement indicates relative improvements over the best baseline in percentage.

- **Chain-of-Thought (CoT)** [Wei *et al.*, 2022]: A reasoning approach for prompting which includes the intermediate steps of reasoning within the prompt.
- **Multiple CoTs (CoT-SC)** [Wang *et al.*, 2023]: A scheme in which multiple CoTs are generated, with the best one being selected as final result.
- **Tree of Thoughts (ToT)** [Yao *et al.*, 2024]: A reasoning approach modeling LLM reasoning process as a tree.

Implementation Details. We choose Llama3-8B-Instruct [AI@Meta, 2024] as the backbone model. SASRec and TALLRec are implemented based on their official code. We implemented PALR by ourselves and LLM reasoning strategies are integrated within the GoT framework. For SLIM, Llama3-8B-Instruct is used instead of ChatGPT to avoid high API costs. We retrieve the 10 most similar items for both GOT4Rec and baseline methods using the faiss library [Douze *et al.*, 2024]. Optimal hyper-parameters for all baselines are carefully selected to ensure the best performance.

Evaluation Metrics. We evaluate performance with hit rate (HR) and normalized discounted cumulative gain (NDCG), reporting HR@K and NDCG@K for $K \in \{5, 10, 20\}$. Each recommended item is evaluated against all other items in the dataset. The average scores of three runs are reported.

4.2 Overall Performance

Table 2 compares our GOT4Rec method with various neural sequential models and LLM reasoning strategies, leading to several key observations:

Performance of Neural & LLM-based Models. Neural and LLM-based sequential models perform relatively modest, though SASRec still outperforms LLM reasoning strategies in some cases, particularly due to its use of self-attention

mechanisms which allow SASRec to effectively model users’ historical behaviors by capturing the transition relationships between items. However, SASRec lacks the ability to comprehend semantic information, limiting its overall performance. As for PALR and TALLRec, their suboptimal performance stems from the absence of strong reasoning capabilities, which are critical for complex recommendation tasks.

Performance of LLM Reasoning Methods. Among LLM reasoning methods, CoT-SC consistently achieves runner-up performance across most datasets, largely because it aggregates and selects the best result from multiple CoT paths, providing a more refined output. On the other hand, CoT, ToT, and SLIM show comparatively lower performance. SLIM, in particular, may suffer from reduced output diversity due to fine-tuning, while ToT’s structural and reasoning path designs appear to be less suitable for sequential recommendation tasks.

Superiority of GOT4Rec. GOT4Rec achieves the state-of-art performances across all datasets. Notably, in the Food dataset, GOT4Rec achieves a relative improvement of 73.93% over CoT-SC in terms of NDCG@10 and 67.49% in terms of HR@5. These significant gains can be attributed to the nature of food product consumption, where users prefer consistent categories or brands and are strongly influenced by short-term needs. This aligns well with the strengths of GOT4Rec, which excels at capturing users’ preferences for certain categories or brands and effectively integrating short-term preference information. This capability allows GOT4Rec to deliver highly relevant recommendations that closely match users’ interests. These findings demonstrate the effectiveness of GOT4Rec in optimizing recommendation tasks by fully exploiting the advanced reasoning capabilities of LLMs and integrating various aspects of user information.

Dataset	Metric	w/o Short-term	w/o Long-term	w/o Collaborative	GOT4Rec
Games	HR@5	0.0819 $\downarrow$ 8.39	0.0871 $\downarrow$ 2.57	0.0774 $\downarrow$ 13.42	0.0894
	HR@10	0.1083 $\downarrow$ 7.20	0.1121 $\downarrow$ 3.94	0.1015 $\downarrow$ 13.02	0.1167
	HR@20	0.1262 $\downarrow$ 7.27	0.1337 $\downarrow$ 1.76	0.1170 $\downarrow$ 14.03	0.1361
	NDCG@5	0.0512 $\downarrow$ 17.55	0.0573 $\downarrow$ 7.73	0.0527 $\downarrow$ 15.14	0.0621
	NDCG@10	0.0597 $\downarrow$ 15.92	0.0654 $\downarrow$ 7.89	0.0606 $\downarrow$ 14.65	0.0710
	NDCG@20	0.0642 $\downarrow$ 13.53	0.0709 $\downarrow$ 6.71	0.0645 $\downarrow$ 15.13	0.0760
Food	HR@5	0.0591 $\downarrow$ 20.35	0.0867 $\uparrow$ 16.85	0.0687 $\downarrow$ 7.41	0.0742
	HR@10	0.0867 $\downarrow$ 10.80	0.0933 $\downarrow$ 4.01	0.0880 $\downarrow$ 9.47	0.0972
	HR@20	0.1003 $\downarrow$ 7.98	0.1063 $\downarrow$ 2.48	0.1013 $\downarrow$ 7.06	0.1090
	NDCG@5	0.0362 $\downarrow$ 26.42	0.0426 $\downarrow$ 13.41	0.0437 $\downarrow$ 11.18	0.0492
	NDCG@10	0.0453 $\downarrow$ 20.11	0.0509 $\downarrow$ 10.23	0.0500 $\downarrow$ 11.82	0.0567
	NDCG@20	0.0487 $\downarrow$ 18.43	0.0543 $\downarrow$ 9.05	0.0534 $\downarrow$ 10.55	0.0597
Home	HR@5	0.0179 $\downarrow$ 6.77	0.0166 $\downarrow$ 13.54	0.0190 $\downarrow$ 1.04	0.0192
	HR@10	0.0284 $\downarrow$ 5.02	0.0292 $\downarrow$ 2.34	0.0270 $\downarrow$ 9.70	0.0299
	HR@20	0.0322 $\downarrow$ 4.45	0.0318 $\downarrow$ 5.64	0.0309 $\downarrow$ 8.31	0.0337
	NDCG@5	0.0102 $\downarrow$ 16.39	0.0103 $\downarrow$ 15.57	0.0114 $\downarrow$ 6.56	0.0122
	NDCG@10	0.0136 $\downarrow$ 13.38	0.0144 $\downarrow$ 8.28	0.0140 $\downarrow$ 10.83	0.0157
	NDCG@20	0.0146 $\downarrow$ 12.57	0.0150 $\downarrow$ 10.18	0.0149 $\downarrow$ 10.78	0.0167

Table 3: Ablation analysis, conducted by retaining different components in GOT4Rec to form variants. The best performance is highlighted in **bold**. The performance difference (%) with GOT4Rec is highlighted with blue and orange. “Short-term”, “Long-term”, and “Collaborative” denote short-term, long-term and collaborative reasoning steps, respectively.

4.3 Ablation Study

We conducted an ablation study to analyze the impact of different components in the GOT4Rec model, with the results in Table 3. Our GOT4Rec consistently outperforms the ablated variants, demonstrating that the full integration of users’ preference information from the short-term, long-term, and collaborative components results in superior recommendation performance. The study also reveals that the importance of each component varies across different datasets, likely reflecting the unique characteristics of each dataset. For instance, collaborative information appears to be the most crucial component in Games dataset. This suggests that when purchasing gaming products, users prioritize categories or brands, making collaborative information from other users with similar interests particularly important. In contrast, short-term preference plays a more significant role in the Food dataset, as users’ recent interactions are often influenced by immediate needs, cravings, or specific dietary goals. This aligns with our inference that users’ recent preferences heavily influence their choices in food products. In the Home dataset, the impact of each component varies, but short-term preference has the least influence. This indicates that long-term preferences or collaborative insights are more critical when users make decisions about home-related items that typically involve more thoughtful consideration.

4.4 Popularity Bias Analysis

To assess recommendation novelty, we calculate EFD@10 and EPC@10 [Vargas and Castells, 2011]. EFD (Expected Free Discovery) represents the expected inverse collection frequency of relevant and seen recommended items. EPC (Expected Popularity Complement) measures the expected

number of seen relevant recommended items that were previously unknown to the user. These two metrics provide a comprehensive assessment of diversity and novelty, helping to balance the recommendation of popular items with long-tail content. Table 4 reports EFD@10 and EPC@10 metrics for CoT, SLIM and GOT4Rec, indicating that GOT4Rec outperforms the baselines in recommending long-tail items. These results support our claim that GOT4Rec mitigates popularity bias by capturing a wider range of information.

Dataset	Metric	CoT	SLIM	GOT4Rec
Games	EFD@10	4.8919	4.1292	5.0929
	EPC@10	0.3885	0.3306	0.3998
Food	EFD@10	6.4721	5.3517	6.8671
	EPC@10	0.4750	0.3923	0.5035
Home	EFD@10	4.1911	3.1141	4.6019
	EPC@10	0.3110	0.2304	0.3450

Table 4: Popularity bias metrics, the higher score indicates higher recommendation novelty. The best score is highlighted in **bold**.

5 Conclusion

In this paper, we propose GOT4Rec, a sequential recommendation method that optimally leverages the reasoning capabilities of LLMs to extract and integrate short-term, long-term, and collaborative user preferences utilizing the graph of thoughts (GoT) framework. Experiments on real-world datasets demonstrate that our GOT4Rec method outperforms existing neural sequential models and LLM reasoning strategies. Further analysis reveals that GOT4Rec effectively integrates multiple pieces of information contained within user sequences for superior recommendations.

References

- [AI@Meta, 2024] AI@Meta. Llama 3 model card. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md, 2024. Accessed: 2024-05-01.
- [Bao et al., 2023] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*, pages 1007–1014, 2023.
- [Besta et al., 2024] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17682–17690, 2024.
- [CohereForAI, 2024] CohereForAI. c4ai-command-r7b-12-2024 model card, 2024. Accessed: 2024-12-15.
- [Dai et al., 2023] Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongxiang Sun, Xiao Zhang, and Jun Xu. Uncovering chatgpt’s capabilities in recommender systems. In *Proceedings of the 17th ACM Conference on Recommender Systems*, pages 1126–1132, 2023.
- [Douze et al., 2024] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library, 2024.
- [Harte et al., 2023] Jesse Harte, Wouter Zorgdrager, Panos Louridas, Asterios Katsifodimos, Dietmar Jannach, and Marios Fragkoulis. Leveraging large language models for sequential recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems, RecSys 2023, Singapore, Singapore, September 18-22, 2023*, pages 1096–1102. ACM, 2023.
- [Hidasi et al., 2016] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based recommendations with recurrent neural networks. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.
- [Hou et al., 2022] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. Towards universal sequence representation learning for recommender systems. In *KDD ’22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*, pages 585–593. ACM, 2022.
- [Hou et al., 2024a] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiushi Chen, and Julian McAuley. Bridging language and items for retrieval and recommendation. *arXiv preprint arXiv:2403.03952*, 2024.
- [Hou et al., 2024b] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian J. McAuley, and Wayne Xin Zhao. Large language models are zero-shot rankers for recommender systems. In *Advances in Information Retrieval - 46th European Conference on Information Retrieval, ECIR 2024, Glasgow, UK, March 24-28, 2024, Proceedings, Part II*, volume 14609 of *Lecture Notes in Computer Science*, pages 364–381. Springer, 2024.
- [Kang and McAuley, 2018] Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*, pages 197–206. IEEE, 2018.
- [Li et al., 2023a] Jiacheng Li, Ming Wang, Jin Li, Jinmiao Fu, Xin Shen, Jingbo Shang, and Julian J. McAuley. Text is all you need: Learning language representations for sequential recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*, pages 1258–1267. ACM, 2023.
- [Li et al., 2023b] Xinhang Li, Chong Chen, Xiangyu Zhao, Yong Zhang, and Chunxiao Xing. E4srec: An elegant effective efficient extensible solution of large language models for sequential recommendation. *arXiv preprint arXiv:2312.02443*, 2023.
- [Lin et al., 2024] Jianghao Lin, Rong Shan, Chenxu Zhu, Kounianhua Du, Bo Chen, Shigang Quan, Ruiming Tang, Yong Yu, and Weinan Zhang. Rella: Retrieval-enhanced large language models for lifelong sequential behavior comprehension in recommendation. In *Proceedings of the ACM on Web Conference 2024*, pages 3497–3508, 2024.
- [Liu et al., 2024] Qijiong Liu, Nuo Chen, Tetsuya Sakai, and Xiao-Ming Wu. Once: Boosting content-based recommendation with both open-and closed-source large language models. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pages 452–461, 2024.
- [MetaLlama, 2024] MetaLlama. Llama-3.2-3b-instruct model card, 2024. Accessed: 2024-12-15.
- [Reimers and Gurevych, 2020] Nils Reimers and Iryna Gurevych. Making monolingual sentence embeddings multilingual using knowledge distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2020.
- [Sanner et al., 2023] Scott Sanner, Krisztian Balog, Filip Radlinski, Ben Wedin, and Lucas Dixon. Large language models are competitive near cold-start recommenders for language- and item-based preferences. In *Proceedings of the 17th ACM Conference on Recommender Systems, RecSys 2023, Singapore, Singapore, September 18-22, 2023*, pages 890–896. ACM, 2023.
- [Team, 2024] Gemma Team. Gemma. 2024.
- [Vargas and Castells, 2011] Saul Vargas and Pablo Castells. Rank and relevance in novelty and diversity metrics for recommender systems. In *Proceedings of the 2011 ACM Conference on Recommender Systems, RecSys 2011, Chicago, IL, USA, October 23-27, 2011*, pages 109–116. ACM, 2011.

- [Wang *et al.*, 2023] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- [Wang *et al.*, 2024] Yuling Wang, Changxin Tian, Binbin Hu, Yanhua Yu, Ziqi Liu, Zhiqiang Zhang, Jun Zhou, Liang Pang, and Xiao Wang. Can small language models be good reasoners for sequential recommendation? In *Proceedings of the ACM on Web Conference 2024*, pages 3876–3887, 2024.
- [Wei *et al.*, 2022] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [Wu *et al.*, 2019] Shu Wu, Yuyuan Tang, Yanqiao Zhu, Liang Wang, Xing Xie, and Tieniu Tan. Session-based recommendation with graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 346–353, 2019.
- [Xi *et al.*, 2024] Yunjia Xi, Weiwen Liu, Jianghao Lin, Xiaoling Cai, Hong Zhu, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, and Yong Yu. Towards open-world recommendation with knowledge augmentation from large language models. In *Proceedings of the 18th ACM Conference on Recommender Systems*, pages 12–22, 2024.
- [Yang *et al.*, 2023] Fan Yang, Zheng Chen, Ziyang Jiang, Eunah Cho, Xiaojiang Huang, and Yanbin Lu. Palr: Personalization aware llms for recommendation. *arXiv preprint arXiv:2305.07622*, 2023.
- [Yang *et al.*, 2024] Juncheng Yang, Zuchao Li, Shuai Xie, Wei Yu, Shijun Li, and Bo Du. Soft-prompting with graph-of-thought for multi-modal representation learning, 2024.
- [Yao *et al.*, 2024] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2024.
- [Yu *et al.*, 2019] Zeping Yu, Jianxun Lian, Ahmad Mahmood, Gongshen Liu, and Xing Xie. Adaptive user modeling with long and short-term preferences for personalized recommendation. In *IJCAI*, volume 7, pages 4213–4219, 2019.
- [Zhang *et al.*, 2022] Mengqi Zhang, Shu Wu, Meng Gao, Xin Jiang, Ke Xu, and Liang Wang. Personalized graph neural networks with attention mechanism for session-aware recommendation. *IEEE Trans. Knowl. Data Eng.*, 34(8):3946–3957, 2022.
- [Zhang *et al.*, 2023] Mengqi Zhang, Shu Wu, Xueli Yu, Qiang Liu, and Liang Wang. Dynamic graph neural networks for sequential recommendation. *IEEE Trans. Knowl. Data Eng.*, 35(5):4741–4753, 2023.
- [Zheng *et al.*, 2022] Yu Zheng, Chen Gao, Jianxin Chang, Yanan Niu, Yang Song, Depeng Jin, and Yong Li. Disentangling long and short-term interests for recommendation. In *Proceedings of the ACM Web Conference 2022*, pages 2256–2267, 2022.
- [Zhou *et al.*, 2020] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *CIKM '20: The 29th ACM International Conference on Information and Knowledge Management, Virtual Event, Ireland, October 19-23, 2020*, pages 1893–1902. ACM, 2020.

A Dataset Statistics

The detailed statistics of the three datasets used in our experiments are presented in Table 5, including the number of users, items, actions (the presence of a review is an action) and time spans covered by each dataset.

Dataset	Users	Items	Actions	Time Span
Games	3,000	12259	26196	1999.11-2023.08
Food	3,000	19256	29321	2003.11-2023.03
Home	3,000	25457	29332	2001.01-2023.03

Table 5: Datasets’ average statistics (after preprocessing).

B LLM Generation Parameters

In table 6, we provide the detailed generation parameters of the LLMs used for generating responses to the given prompts. By outlining these parameters, we ensure reproducibility and offer insights into the configuration choices that impact model performance.

Parameter	Value
temperature	0.6
top-k	40
top-p	0.8
max sequence length	4096
presence penalty	0.02
frequency penalty	0.02
repetition penalty	1.02

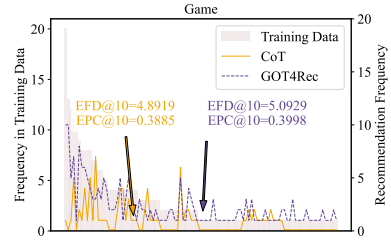
Table 6: LLM parameters for text generation.

C Popularity Bias Analysis

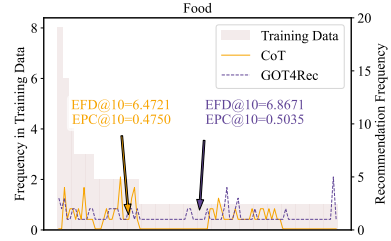
In Figure 4, we sort the items in all three datasets based on their frequency in the training set (i.e., popularity) and draw lines to illustrate each item’s frequency in the results of CoT and GOT4Rec. This visualization highlights the distribution of recommendations across both popular and tail items. It is evident from the figure that GOT4Rec demonstrates a significant advantage in recommending tail items. Additionally, GOT4Rec achieves a broader coverage of items, indicating its ability to better capture and utilize the long-tail distribution.

D Generalizability of the Method

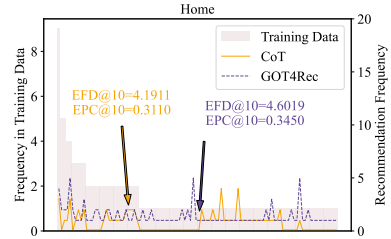
To evaluate the generalizability of our method, we conduct experiments using a variety of alternative LLM backbones. Specifically, we test Llama3.2-3B-Instruct [MetaLlama, 2024], Gemma2-9B-Instruct [Team, 2024] and Command-R-7B [CohereForAI, 2024], which represent a diverse set of model architectures and parameter scales. These backbones were selected to provide a robust and comprehensive evaluation of our method. The results, presented in Table 7, demonstrate that our method remains effective on the Food dataset, but exhibits uneven performance on the other two datasets. In both the Food and Home datasets, the HRs are generally above the baselines, but the NDCGs are noticeably



(a) Popularity bias in Games dataset.



(b) Popularity bias in Food dataset.



(c) Popularity bias in Home dataset.

Figure 4: Analysis of popularity bias, items in the dataset are sorted by their frequency. Compared to CoT, GOT4Rec demonstrates a more consistent ability to recommend long-tail items.

lower, indicating that while relevant items are being recommended, their ranking positions are suboptimal. We speculate that these discrepancies stem from inherent differences in the LLMs themselves, as their knowledge bases and reasoning abilities vary significantly, which can impact their ability to process and prioritize diverse types of information.

Dataset	Metric	SOTA-Baselines	Llama3-8B	Llama3.2-3B	Gemma2-9B	Command-R-7B
Games	HR@5	0.0776	0.0894 ^{↑15.21}	0.0623 ^{↓19.72}	0.0655 ^{↓15.59}	0.0604 ^{↓22.16}
	HR@10	0.1013	0.1167 ^{↑15.20}	0.1041 ^{↑2.76}	0.1152 ^{↑13.72}	0.1023 ^{↑0.99}
	HR@20	0.1347	0.1361 ^{↑1.04}	0.1384 ^{↑2.75}	0.1327 ^{↓1.48}	0.1289 ^{↓4.31}
	NDCG@5	0.0528	0.0621 ^{↑17.61}	0.0397 ^{↓24.81}	0.0392 ^{↓25.76}	0.0376 ^{↓28.79}
	NDCG@10	0.0586	0.0710 ^{↑21.16}	0.0531 ^{↓9.39}	0.0548 ^{↓6.48}	0.0524 ^{↓10.58}
	NDCG@20	0.0655	0.0760 ^{↑16.03}	0.0618 ^{↓5.65}	0.0647 ^{↓1.22}	0.0611 ^{↓6.72}
Food	HR@5	0.0443	0.0742 ^{↑67.49}	0.0566 ^{↑27.77}	0.0472 ^{↑6.55}	0.0517 ^{↑16.70}
	HR@10	0.0597	0.0972 ^{↑62.81}	0.0747 ^{↑25.13}	0.0686 ^{↑14.91}	0.0701 ^{↑17.42}
	HR@20	0.0753	0.1090 ^{↑44.75}	0.0980 ^{↑30.15}	0.1114 ^{↑47.94}	0.1043 ^{↑38.51}
	NDCG@5	0.0286	0.0492 ^{↑72.03}	0.0348 ^{↑21.68}	0.0308 ^{↑7.69}	0.0321 ^{↑12.24}
	NDCG@10	0.0326	0.0567 ^{↑73.93}	0.0409 ^{↑25.46}	0.0378 ^{↑15.95}	0.0388 ^{↑19.02}
	NDCG@20	0.0366	0.0597 ^{↑63.11}	0.0471 ^{↑28.69}	0.0491 ^{↑34.15}	0.0462 ^{↑26.23}
Home	HR@5	0.0133	0.0192 ^{↑44.36}	0.0175 ^{↑31.58}	0.0144 ^{↑8.27}	0.0163 ^{↑22.56}
	HR@10	0.0223	0.0299 ^{↑34.08}	0.0281 ^{↑26.01}	0.0215 ^{↓3.59}	0.0231 ^{↑3.59}
	HR@20	0.0270	0.0337 ^{↑24.81}	0.0330 ^{↑22.22}	0.0341 ^{↑26.30}	0.0328 ^{↑21.48}
	NDCG@5	0.0097	0.0122 ^{↑25.77}	0.0087 ^{↓10.31}	0.0071 ^{↓26.80}	0.0089 ^{↓8.25}
	NDCG@10	0.0109	0.0157 ^{↑44.04}	0.0118 ^{↑8.26}	0.0082 ^{↓24.77}	0.0103 ^{↓5.50}
	NDCG@20	0.0134	0.0167 ^{↑24.63}	0.0131 ^{↓2.24}	0.0110 ^{↓17.91}	0.0126 ^{↓5.97}

Table 7: Results of different LLM backbones, the best score is highlighted in **bold**. The performance difference (%) with SOTA baselines is highlighted with **blue** and **orange**.